WATER TANK MONITER

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of the
Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

Submitted By

RAVI SUNIL PRAJAPATI

Roll No. 311

Seat No. 510515438

Under the Esteemed Guidance of

ANAKEET NENA

Asst. Professor

DEPARTMENT OF BSc. COMPUTER SCIENCE

PRAHLADRAI DALMIA LIONS COLLEGE OF
COMMERCE & ECONOMICS

NAAC Re-accredited with 'A' Grade (III Cycle) ISO 21001:2018 Certified

(Affiliated to University of Mumbai)

MUMBAI, 400064.

2026 - 2027

Prahladrai Dalmia Lions College of Commerce & Economics

(Government Aided & Affiliated to University of Mumbai & AICTE,New Delhi)
NAAC Re-accredited with 'A' Grade (III Cycle) ISO 21001:2018 Certified

University of Mumbai Recognised Research Centre in Accountancy,Commerce & Business Economics

A Government Approved Hindi Linguistic Minority Institute

DEPARTMENT OF BSc. COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled, **"WATER TANK MONITER"**, is bonafide work of Mr. **RAVI SUNIL PRAJAPATI** bearing Seat No: **311** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in COMPUTER SCIENCE from University of Mumbai.

30/10/26

Internal Guide

30/10/26

Coordinator

30/10/26

External Examiner

30/10/26
Date:

College Seal

DECLARATION

I the undersigned **RAVI PRAJAPATI** hereby declare that the project entitled, **"WATER TANK MONITER"** done at **Prahladrai Dalmia Lions College of Commerce & Economics**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** to be submitted as final semester project as part of our curriculum.

Signature

(RAVI PRAJAPATI)

Acknowledgement

To list who all have helped me is difficult because they are so numerous and the depths so enormous.

I would like to acknowledge the following as being idealistic channels and fresh dimensions in the completion of this project. I take this opportunity to thank the **University of Mumbai** for giving me the chance to do this project.

I would like to thank my **Principal Prof. (Dr.) Digambar N. Ganjewar** and our **Vice-Principal Ms. Subhashini Naikar** for providing the necessary facilities required for completion of this project. I take this opportunity to thank our **Coordinator Dr. Sneha Hathi** for her continuous support and guidance.

I would also like to express my sincere gratitude towards my project guide **Mr. Anakeet Nena** whose guidance and care made the project successful.

I would like to thank my **College Library**, for having provided various references Books and magazines related to my project.

Lastly, I would like to thank each and every person who directly or indirectly helped me in the completion of the project especially **my Parents and Peers** who supported me throughout my project.

RAVI PRAJAPATI

ROLES AND RESPONSIBILITIES

I **Ravi Prajapati** take the responsibility of Completing the project pertaining to which we are members. Our Role in this Project involves the development of the whole system from scratch. All the Roles and Responsibilities will be fulfilled by us within the given time:

Roles and Responsibilities

- **Frontend Module :** The UI Designing & Frontend development.
- **UI / UX Design :** Designing of website.
- **Analytics & Dashboard Module :** This module will be used for users dashboard stats display.
- **Course & Content Module :** This module is responsible for content of the software.
- **Backend Module :** The Backend design.
- **Authentication Module:** This module will use to authenticate the user login credentials.
- **Progress Tracking & Gamification Module :** This module will be used to track the daily progress of the user.
- **Audio & Pronunciation Module :** This module includes the AI audio & pronunciation for gamified environments.
- **Monetization & Premium Module :** This module includes a premium unlimited heart feature of paid certifications.

ABSTRACT

The **Water Tank Monitor** is a system designed to monitor the water level in a tank and help prevent water overflow and water shortages. The main objective of this project is to reduce manual effort and provide an easy and efficient method for checking the water level. The system uses a water-level sensor to detect the amount of water present in the tank and sends the readings to a microcontroller, such as Arduino, for processing.

The system identifies whether the tank is empty, partially filled, or completely full. The current water level and status can be displayed using an LCD display, while LED indicators provide visual information about the tank condition. An alert system can also be implemented to notify users when the water level is too low or the tank is full. Depending on the hardware and programming, the system can be extended to support automatic water pump control.

The Water Tank Monitor aims to save water, reduce human intervention, and improve water management in homes, buildings, and other facilities. It provides a simple, cost-effective, and reliable solution for water-level monitoring. In the future, the system can be enhanced with a web dashboard, database storage, mobile notifications, and remote monitoring to make water management more convenient and efficient.

Contents

PURPOSE OF THE PROJECT 1

SOFTWARE REQUIREMENT SPECIFICATION (SRS) 2

Overall Description 2

1. System Features 3

2. Functional Requirements 4

3. Non -Functional Requirements 5

External Interface requirements 6

COMPLETE UML SET 7

1. Use Case Diagram 7

2. ER Diagram 9

3. Activity Diagram 11

4. Class Diagram 13

5. Sequence Diagram 15

6. Deployment Diagram 17

SYSTEM ARCHITECTURE DESIGN 19

1. Frontend Architecture 19

2. Backend Architecture 21

3. Database Scheme Design 23

4. API Structure 25

5. Security Architecture 27

WORKING HARDWARE 29

1. Working 29

2. Coding 30

PURPOSE OF THE PROJECT

The purpose of the Water Tank Monitor project is to develop a simple and efficient system for monitoring the water level in a tank automatically. Water is an essential resource, and its unnecessary wastage can occur when a tank is filled beyond its required level or when the water level is not properly monitored. The proposed system helps users keep track of the water level and provides an indication when the tank reaches different levels.

The project uses an Arduino Uno as the main control unit and an HC-SR04 ultrasonic sensor to measure the distance between the sensor and the water surface. Based on the measured distance, the system determines the approximate water level inside the tank. A 16×2 LCD display is used to show the water-level information, making it easy for the user to understand the current condition of the tank.

The system also uses LED indicators to represent different water levels. For example, a green LED can indicate a normal water level, a yellow LED can indicate a medium level, and a red LED can indicate that the tank is nearly full. A buzzer can provide an audio warning when the water reaches a critical or high level. These indicators allow the user to identify the tank condition quickly without manually checking the tank.

The main purpose of this project is to reduce water wastage, provide continuous water-level monitoring, and improve the convenience and safety of water management. The system can be useful in homes, colleges, offices, and other places where water tanks are regularly used. It also demonstrates how sensors, microcontrollers, displays, LEDs, and buzzers can be integrated to create a practical IoT and embedded-system-based monitoring solution.

Overall, the Water Tank Monitor provides a low-cost and user-friendly approach to water-level monitoring. It helps users make better use of available water, prevents overflow, and provides timely indications about the condition of the tank.

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Overall Description

a. User Characteristics

i. Water Tank User

- Monitors the current water level in the tank using the Water Tank Monitor system.
- Views the water level and related information displayed on the LCD screen.
- Responds to LED and buzzer alerts when the water level reaches a high or critical level.

ii. Administrator

- Accesses basic system settings and monitoring functions.
- Checks the sensors and other components when required.
- Ensures proper system operation and accurate water-level indication.

b. Constraints

i. The system requires a continuous power supply for the Arduino and connected components.

ii. The accuracy of water-level measurement depends on the proper placement and operation of the ultrasonic sensor.

iii. The system is designed for a specific tank size and may require adjustment for different tank dimensions.

iv. The system may not provide accurate readings if the water surface is highly disturbed or the sensor is not properly positioned.

c. Assumptions

i. The ultrasonic sensor is installed correctly inside the water tank.

ii. The Arduino and other components receive a stable power supply.

iii. The water level is measured accurately under normal conditions.

iv. The user can understand the LCD, LED, and buzzer indications.

1. System Features

a. Water Level Detection

i. Measures the water level using an ultrasonic sensor.
ii. Continuously monitors the tank level.
iii. Provides accurate level information to the system.

b. LCD Display

i. Displays the current water level.
ii. Shows the status of the tank.
iii. Provides easy-to-read information to the user.

c. LED Indicators

i. Green LED indicates a normal water level.
ii. Yellow LED indicates a medium water level.
iii. Red LED indicates a high water level.

d. Buzzer Alert

i. Gives an alert when the water level is high.
ii. Warns the user when the tank is nearly full.
iii. Helps prevent water overflow.

e. Arduino Control

i. Processes data received from the ultrasonic sensor.
ii. Controls the LCD, LEDs, and buzzer.
iii. Performs continuous monitoring of the tank.

f. Power and Operation

i. Operates using a suitable power supply.
ii. Provides continuous monitoring during operation.
iii. Designed for simple and user-friendly operation.

2. Functional Requirements

a. FR - Water Level Measurements

i. The system shall measure the distance between the sensor and water surface.
ii. The system shall calculate the approximate water level.
iii. The system shall continuously monitor the water level.

b. FR - LCD Display

i. The system shall display the current water level.
ii. The LCD shall show the tank status.
iii. The display shall update according to the water level.

c. FR – LED Indication

i. The green LED shall indicate a normal water level.
ii. The yellow LED shall indicate a medium water level.
iii. The red LED shall indicate a high water level.

d. FR - Buzzer Alert

i. The buzzer shall turn on when the water level is high.
ii. The system shall provide an audio warning before overflow.
iii. The buzzer shall turn off when the level returns to normal.

e. FR – Arduino Processing

i. The Arduino shall receive data from the ultrasonic sensor.
ii. The Arduino shall process the measured water-level data.
iii. The Arduino shall control the LCD, LEDs, and buzzer according to the water level.

3. Non -Functional Requirements

a. NFR – Performance

i. The system shall provide quick water-level readings.
ii. The system shall continuously monitor the tank.
iii. The system shall respond quickly to level changes.

b. NFR – Reliability

i. The system shall work consistently during operation.
ii. The sensor shall provide reliable level measurements.
iii. The system shall minimize incorrect alerts.

c. NFR – Usability

i. The system shall be simple and easy to use.
ii. The LCD shall display information clearly.
iii. LED and buzzer alerts shall be easy to understand.

d. NFR – Maintainability

i. The system shall be easy to maintain.
ii. Components shall be replaceable when required.
iii. The system shall allow easy checking of connections.

e. NFR – Safety

i. The system shall provide an alert before water overflow.
ii. Electrical connections shall be properly insulated.
iii. The system shall operate with a suitable power supply.

External Interface requirements

a. User Interface

i. The LCD shall display the current water level.
ii. LEDs shall indicate different water-level conditions.
iii. The buzzer shall provide an audio alert when required.

b. Hardware Interface

i. The ultrasonic sensor shall be connected to the Arduino.
ii. The LCD shall be connected to display water-level information.
iii. LEDs and buzzer shall be connected to the Arduino for alerts.

c. Software Interface

i. The Arduino program shall process sensor readings.
ii. The program shall control the LCD, LEDs, and buzzer.
iii. The software shall update the system status continuously.

d. Communication Interface

i. The system shall communicate between the sensor and Arduino.
ii. The Arduino shall send output signals to connected components.
iii. The system shall provide real-time status through the display and alerts.

COMPLETE UML SET

1. Use Case Diagram

A Use Case Diagram is a visual representation of how users and external entities interact with a system to achieve specific goals. It is part of the Unified Modeling Language (UML) and focuses on what a system does from the user's perspective rather than how it works internally.

a. Key components

i. **Actors:** The users or external entities that interact with the system. They are represented as stick figures outside the system boundary.
ii. **Use Case:** The specific functions or tasks performed by the system. They are represented as ovals and labeled with action names.
iii. **System Boundary:** A rectangle enclosing all use cases, defining the scope and limit of the system.

b. Entities Used

i. Administrator
ii. Water Tank User
iii. LCD Display
iv. LED Indicators
v. Sensor Data

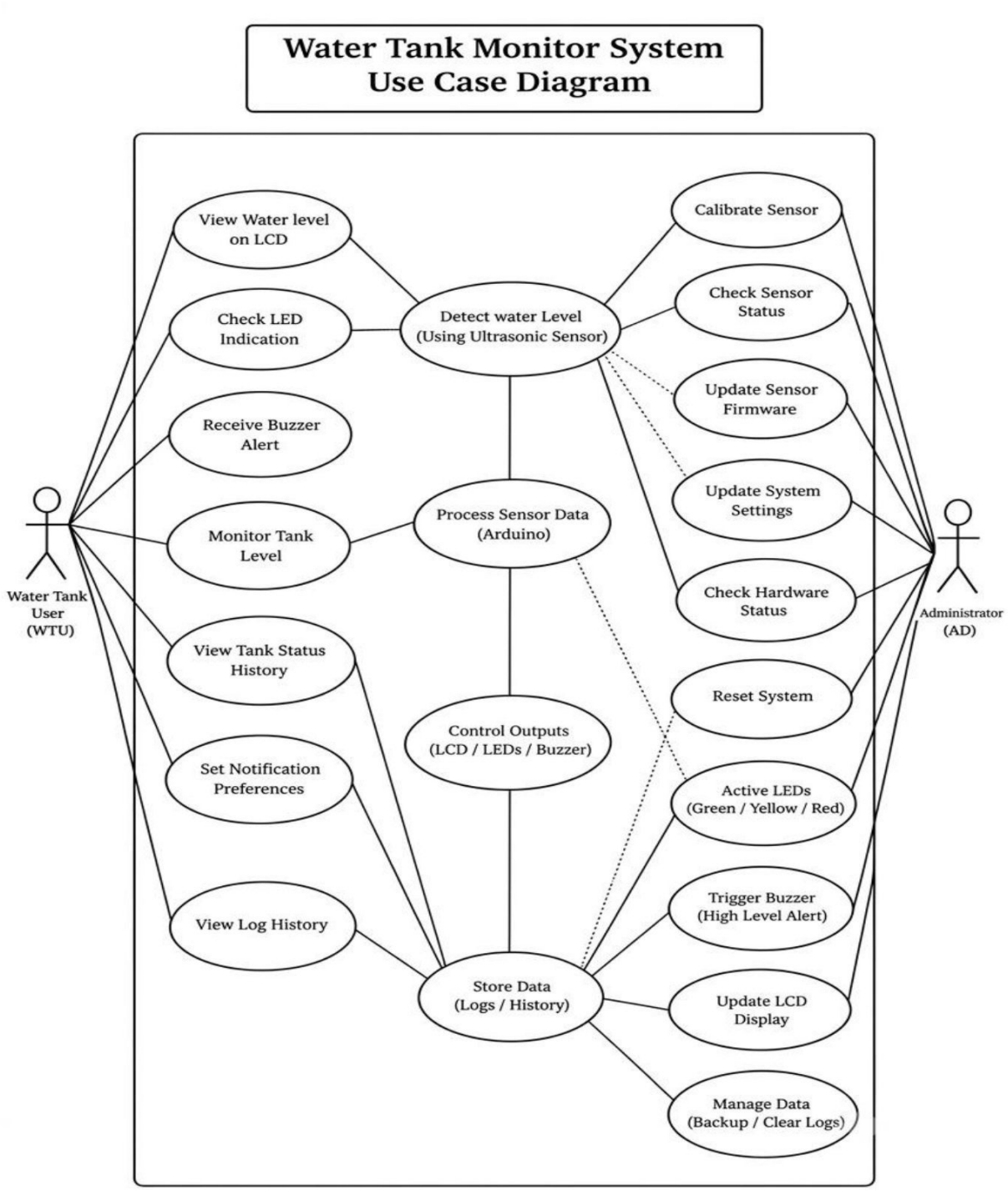


Figure 1: Use Case Diagram

2. ER Diagram

An Entity Relationship (ER) Diagram is a visual representation of the entities in a system and the relationships between them. It is used in database design to show how data is organized, stored, and connected within the Water Tank Monitor System.

a. Key Components

i. **Entities:** Objects or items about which data is stored, such as Water Tank, Sensor, Water Level, and Water Pump. They are represented by rectangles.

ii. **Attributes:** The properties or details of an entity, such as Tank ID, Water Level, Sensor ID, and Timestamp. They are represented by ovals.

iii. **Relationships:** The connections between two or more entities. They are represented by diamonds.

iv. Primary Key: A unique attribute used to identify each record, such as Tank ID or Sensor ID.

v. **Foreign Key:** An attribute that connects related records between different entities.

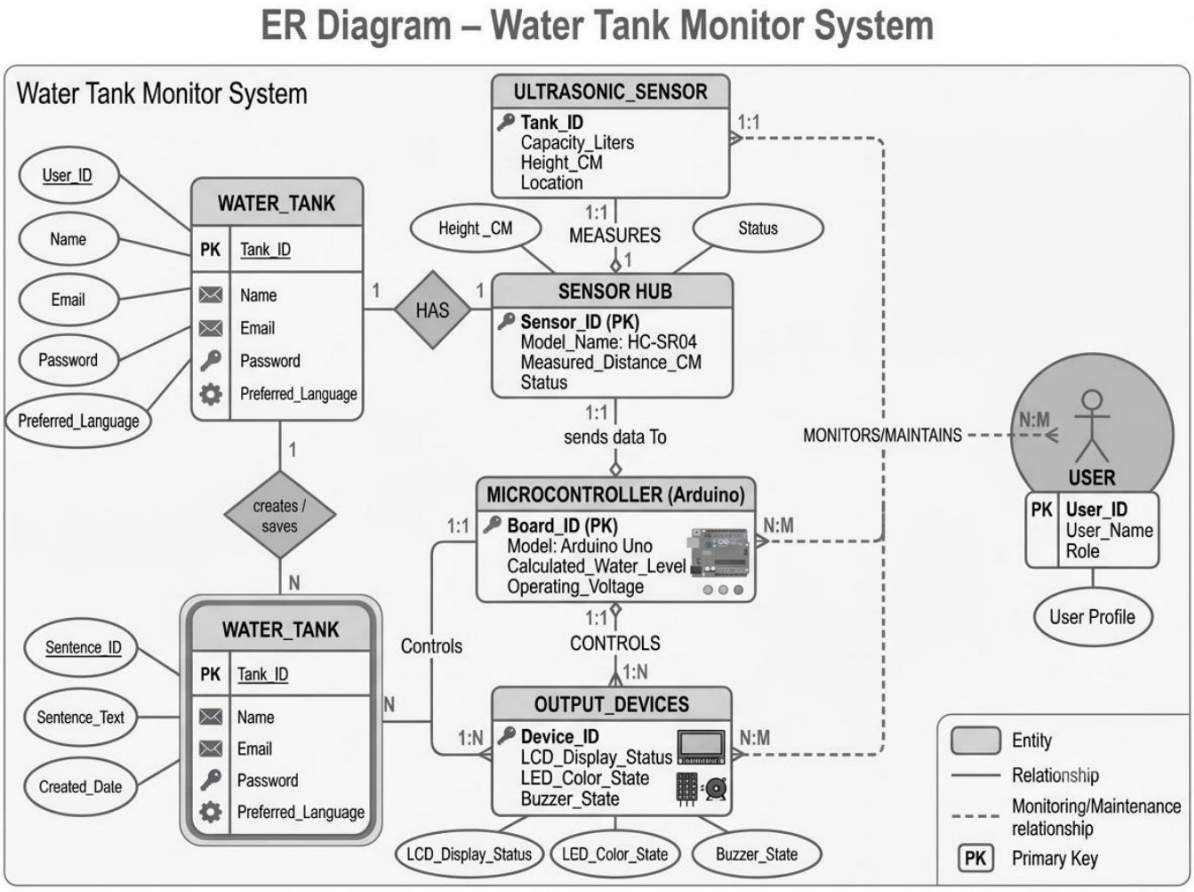


Figure 2: ER Diagram

3. Activity Diagram

An Activity Diagram is a visual representation of the workflow or sequence of activities performed in a system. It is part of the Unified Modeling Language (UML) and describes the step-by-step process of how the Water Tank Monitor System operates, from starting the system to detecting the water level and generating alerts.

a. Key Components

i. **Initial Node:** Represents the starting point of the system. It is shown as a filled black circle.

ii. **Activities:** Represent the tasks performed by the system, such as reading sensor data, processing water-level readings, and updating the LCD.

iii. **Decision Node:** Represents a condition where the system chooses between different paths, such as low, normal, or full water levels. It is shown as a diamond.

iv. **Control Flow:** Arrows indicate the direction and sequence of activities.

v. **Final Node:** Represents the end of a process. It is shown as a filled circle surrounded by another circle.

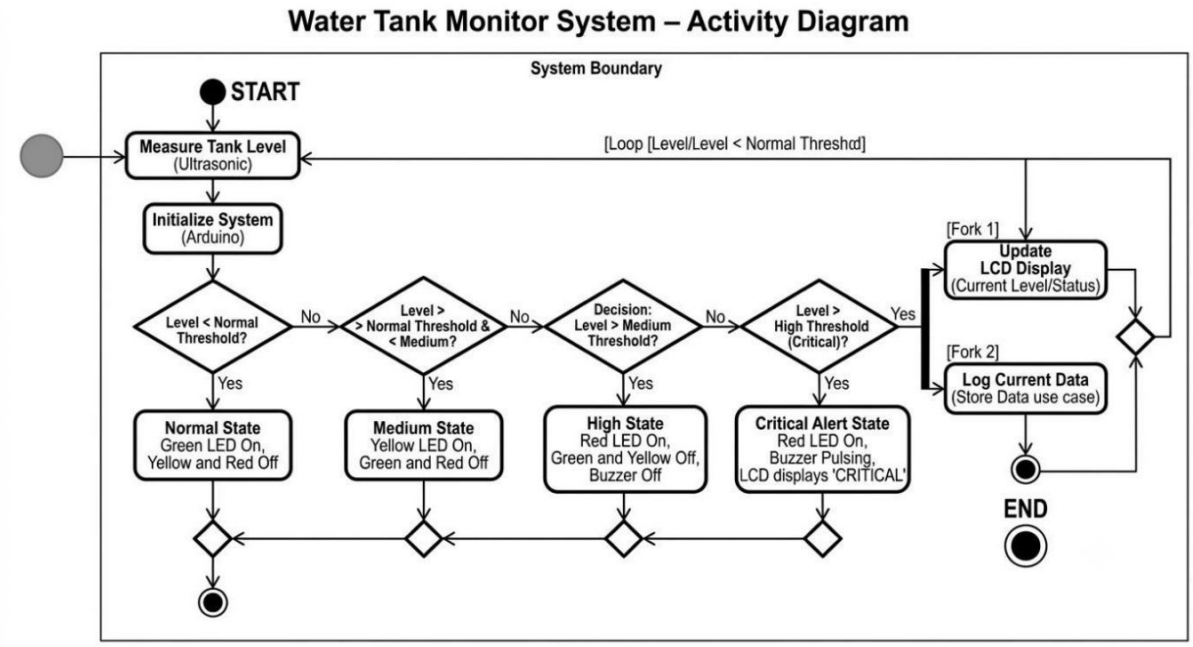


Figure 3:Activity Diagram

4. Class Diagram

A Class Diagram is a structural diagram in UML that represents the classes, attributes, methods, and relationships of a system. It shows how different classes interact with each other in the Water Tank Monitor system.

a. Key Components

i. . Classes: Represent the main objects of the system, such as WaterTank, Sensor, LCDDisplay, LEDIndicator, and Administrator.

ii. Attributes: Define the properties of each class, such as water level, sensor status, and tank capacity.

iii. iii. Methods: Define the functions performed by each class, such as measuring water level, displaying status, and generating alerts.

iv. iv. Relationships: Show how different classes are connected and interact with one another.

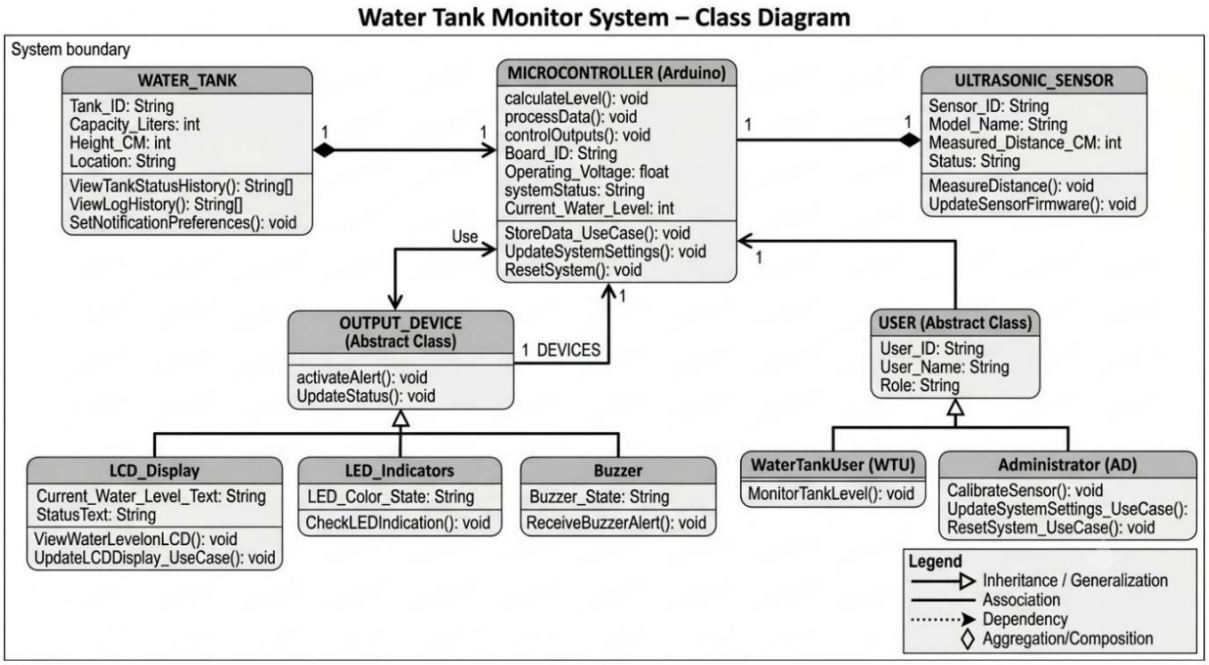


Figure 4: Class Diagram

5. Sequence Diagram

A Sequence Diagram is a UML diagram that shows the sequence of interactions between users and system components over time. It explains how messages are exchanged between different entities in the Water Tank Monitor system.

a. Key Components

i. **Actors:** The Administrator who monitors the water tank.

ii. Objects: Water Tank Sensor, Controller, LCD Display, and LED Indicators.

iii. Messages: Requests and responses exchanged between system components.

iv. Lifelines: Vertical dashed lines representing the existence of each object over time.

v. Activation Bars: Show when an object is performing an operation.

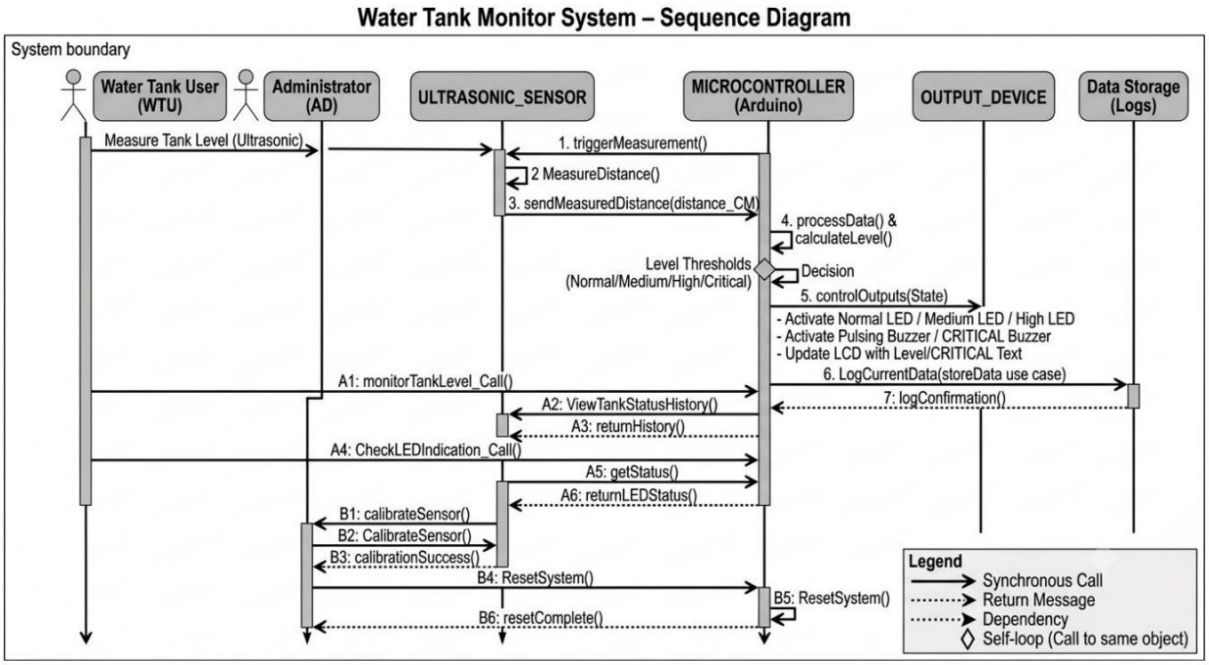


Figure 5:Sequence diagram

6. Deployment Diagram

A Deployment Diagram is a UML diagram that represents the physical hardware and software components of a system. It shows how the software is deployed on hardware devices in the Water Tank Monitor system.

a. Key Components

i. **Nodes:** Physical devices or hardware on which the system operates

ii. Devices: Hardware components such as sensors, microcontrollers, LCD displays, and LED indicators.

iii. Artifacts: Software programs or data deployed on the devices.

iv. Communication Paths: Connections between devices that allow them to exchange data.

b. Entities Used

i. Water Tank

ii. Water Level Sensor

iii. Microcontroller (Arduino)

iv. LCD Display

v. LED Indicators

vi. vi. Power Supply

vii. vii. Administrator

viii. Bottom of Form

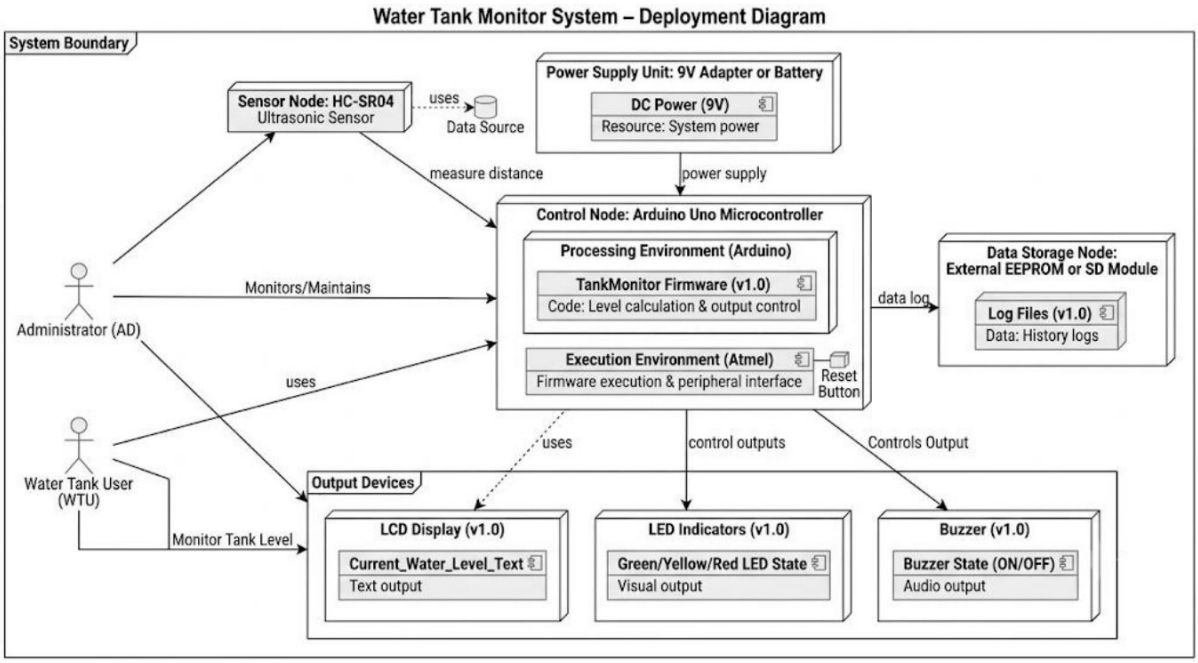


Figure 6: Deployment Diagram

SYSTEM ARCHITECTURE DESIGN

1. Frontend Architecture

Frontend Architecture is the user interface of the Water Tank Monitor system. It allows the administrator or user to check the current water level in the tank easily. It displays important information such as water level percentage, tank status, and warning messages. The frontend can be designed using HTML, CSS, and JavaScript for a web-based system, or it can use an LCD display and LED indicators for a hardware-based system.

The main purpose of the frontend is to present water-level information in a simple and understandable format. It can show whether the tank is empty, half-full, or full. It can also display alerts when the water level becomes too low or when the tank reaches its maximum capacity. These features help users understand the tank's condition without checking it manually.

The frontend improves the usability and monitoring of the system. It saves time, reduces manual effort, and helps users respond quickly to water-level changes. It can also be extended with a dashboard, graphical water-level indicators, notifications, and remote monitoring features if the system supports internet connectivity.

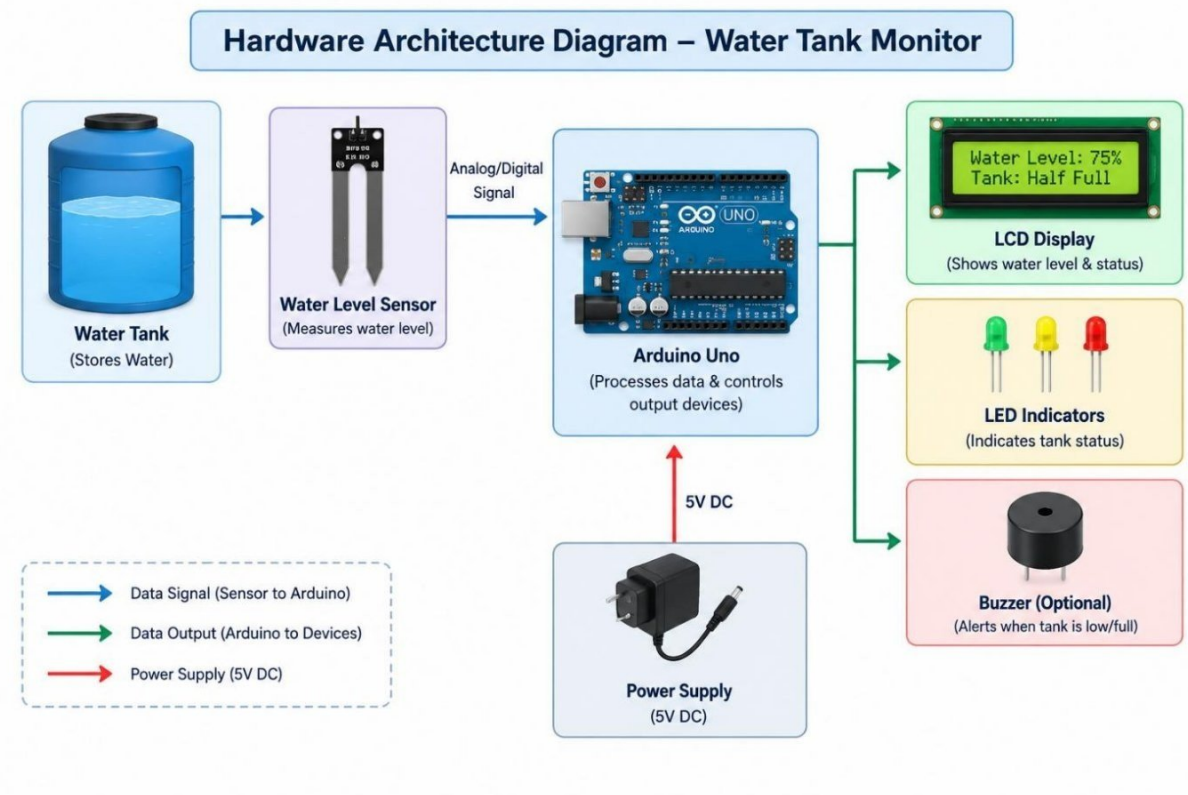


Figure 7:Frontend Architecture

2. Backend Architecture

Backend Architecture is the internal processing part of the Water Tank Monitor system. It receives water-level readings from the sensor and processes them using a microcontroller, such as Arduino. The backend checks the sensor values and determines the current water level in the tank. It acts as the main control unit that manages the system's operations.

The backend uses control logic to identify whether the tank is empty, partially filled, or full. According to the sensor readings, it sends the appropriate information to the LCD display and controls the LED indicators. It can also activate an alarm or a water pump automatically when specific water-level conditions are reached, provided the required hardware is connected and programmed.

The main purpose of the backend is to ensure that the system works accurately and efficiently. It reduces the need for manual water-level checking and helps prevent water overflow and unnecessary water shortages. Additional features such as data storage, usage history, automatic pump control, and mobile notifications can also be implemented to make the Water Tank Monitor more useful and reliable.

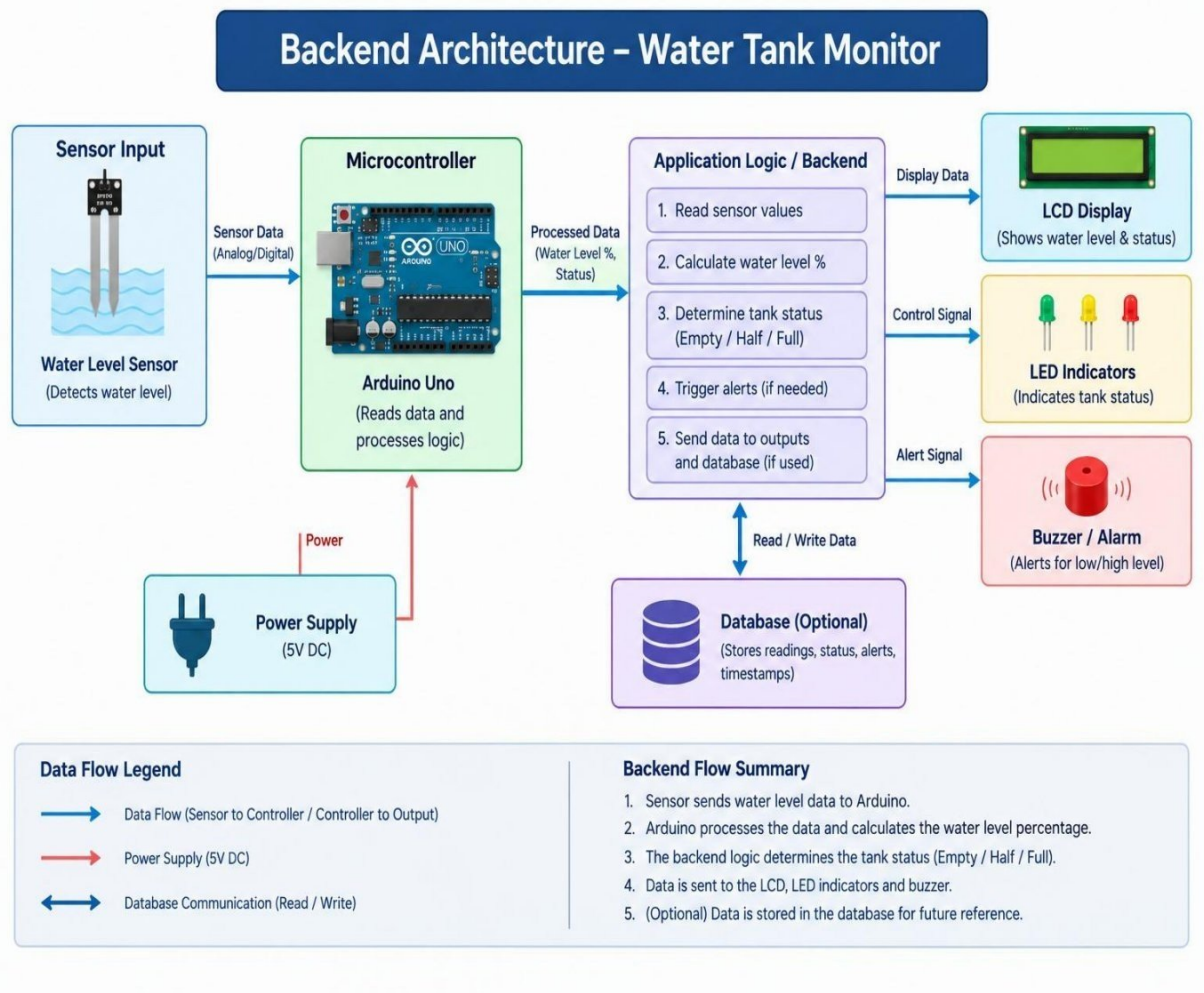


Figure 2: Backend Architecture

3. Database Scheme Design

Database Schema Design defines how data is stored, organized, and managed in the Water Tank Monitor system. It contains information about the water tank, sensor readings, water level status, and alerts. The d

atabase helps store and retrieve information whenever required. It can be designed using MySQL or another database management system if the project needs data storage.

The database can contain tables such as WaterTank, Sensor, WaterLevel, and Alerts. The WaterTank table stores the tank ID, tank name, and capacity. The Sensor table stores sensor ID, sensor type, and sensor status. The WaterLevel table stores the water level percentage, measured value, and date and time of each reading. The Alerts table stores alert ID, alert message, alert type, and the date and time when the alert was generated.

The main purpose of database schema design is to maintain accurate and organized records of water-level information. It helps users check previous readings, monitor changes in water levels, and identify overflow or low-water situations. The stored data can also be used to generate reports, analyze water consumption patterns, and improve water management. If the project uses only an Arduino, LCD, and LED indicators without data storage, a database is optional; it is mainly useful when historical records or remote monitoring are required.

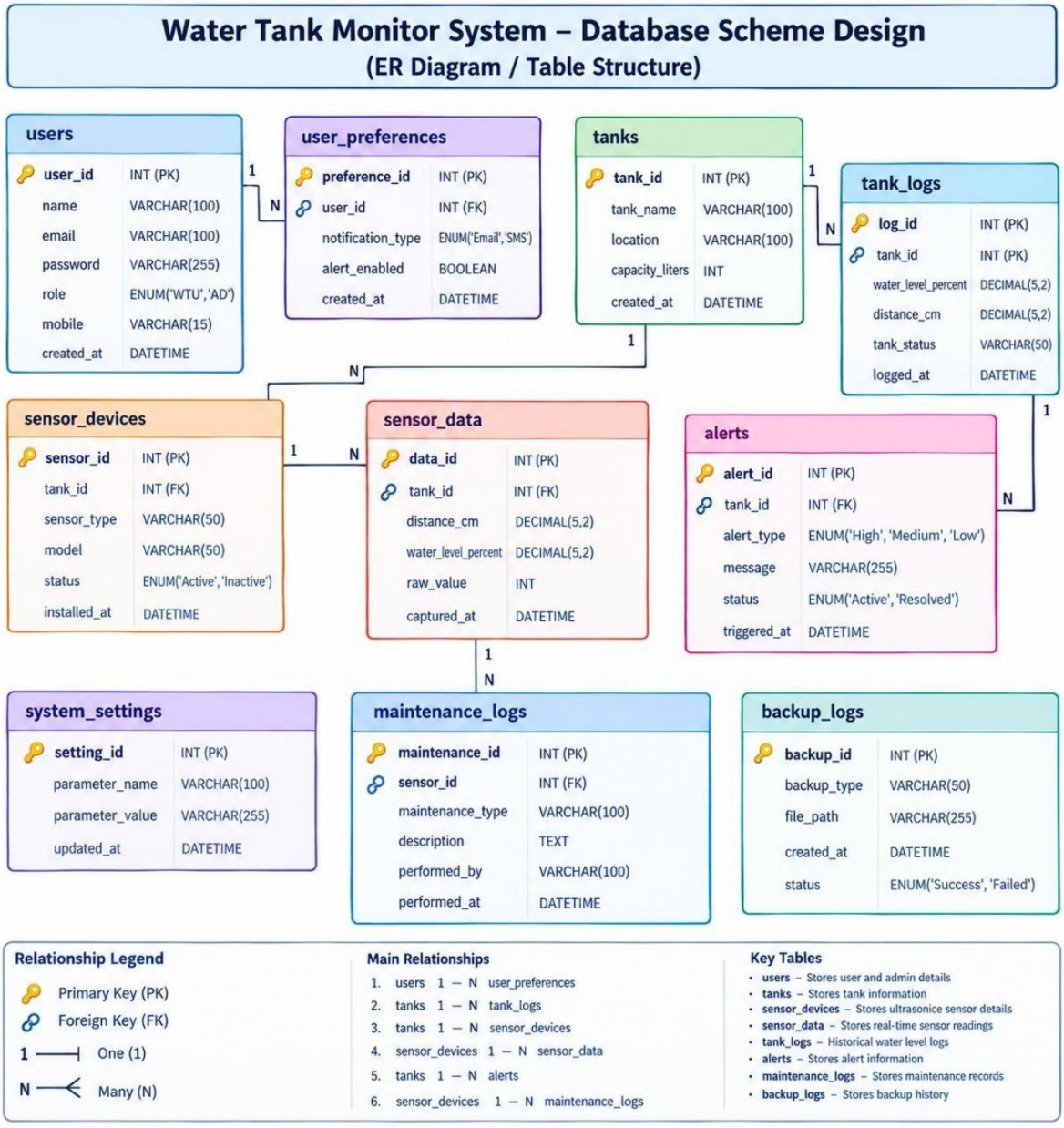


Figure 2:Database Scheme Design

4. API Structure

API Structure defines how the frontend and backend communicate with each other in the Water Tank Monitor system. An API (Application Programming Interface) allows different software components to exchange data and perform operations. In this system, the API receives water-level information from the backend or sensor-connected controller and provides it to the frontend for display. It can be implemented using REST APIs and HTTP methods when the system supports network connectivity.

The API can include endpoints such as GET /api/waterlevel to retrieve the current water level, GET /api/tank/status to check whether the tank is empty, half-full, or full, and GET /api/alerts to retrieve water-level alerts. A POST /api/waterlevel endpoint can be used to send new sensor readings to the backend. The API can exchange data in JSON format, including the tank ID, water level percentage, status, and timestamp.

The main purpose of the API is to provide smooth communication between the frontend, backend, and database. It helps display updated water-level information, retrieve previous readings, and send alerts when the tank is nearly empty or full. It can also support future features such as mobile applications, remote monitoring, and automatic pump control. Proper authentication, input validation, and secure communication should be used to protect the system.

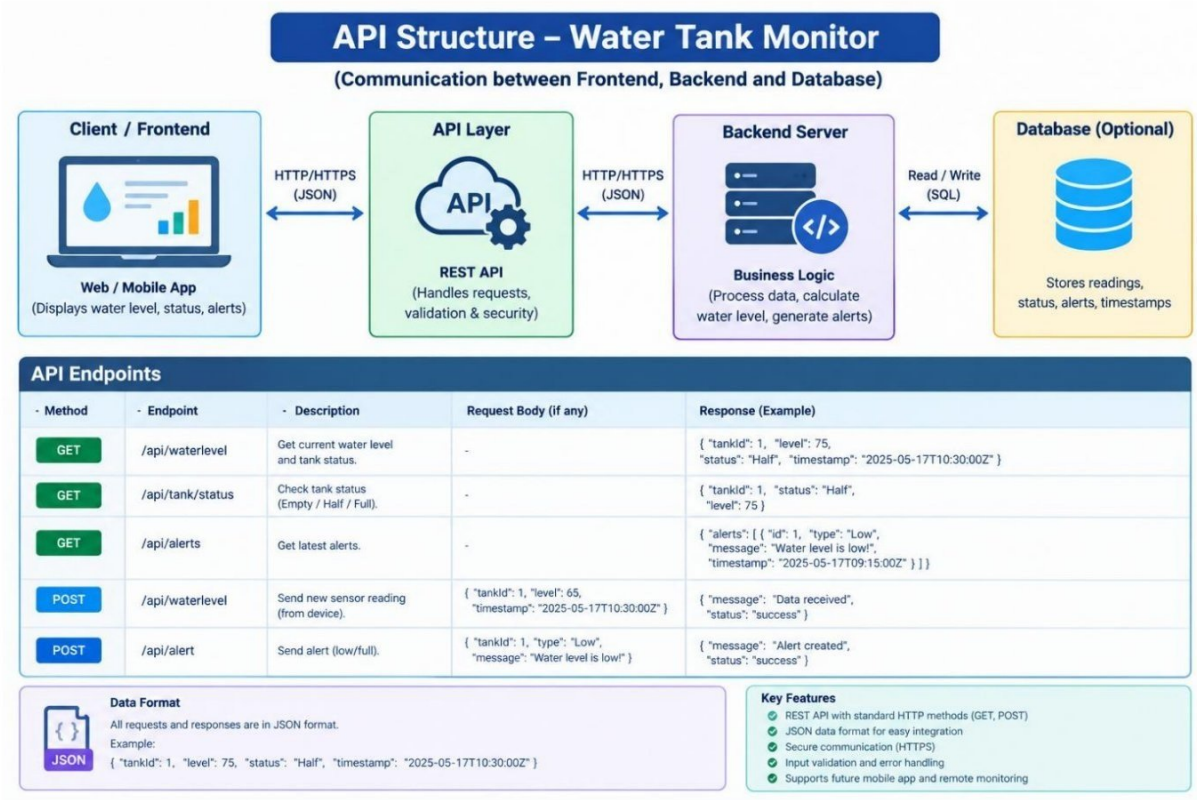


Figure 3: Api Structure

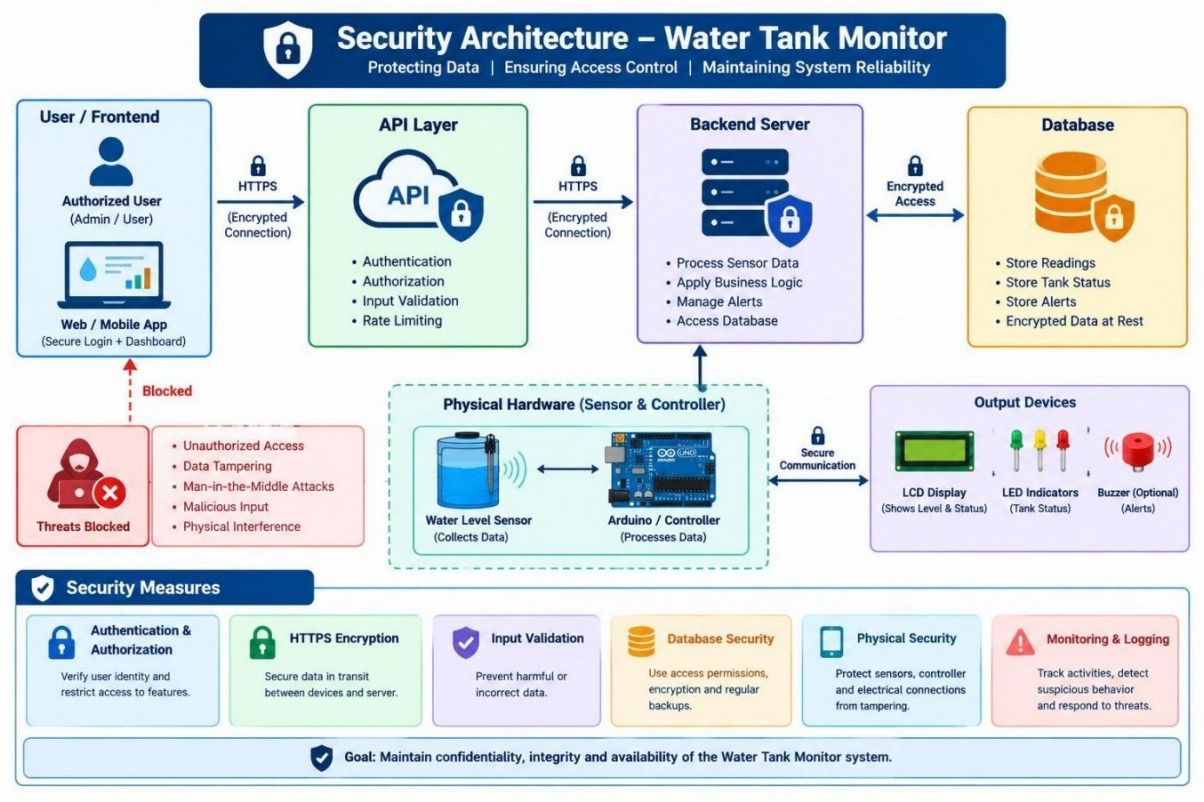


Figure:5 Security Architecture

WORKING HARDWARE

1. Working

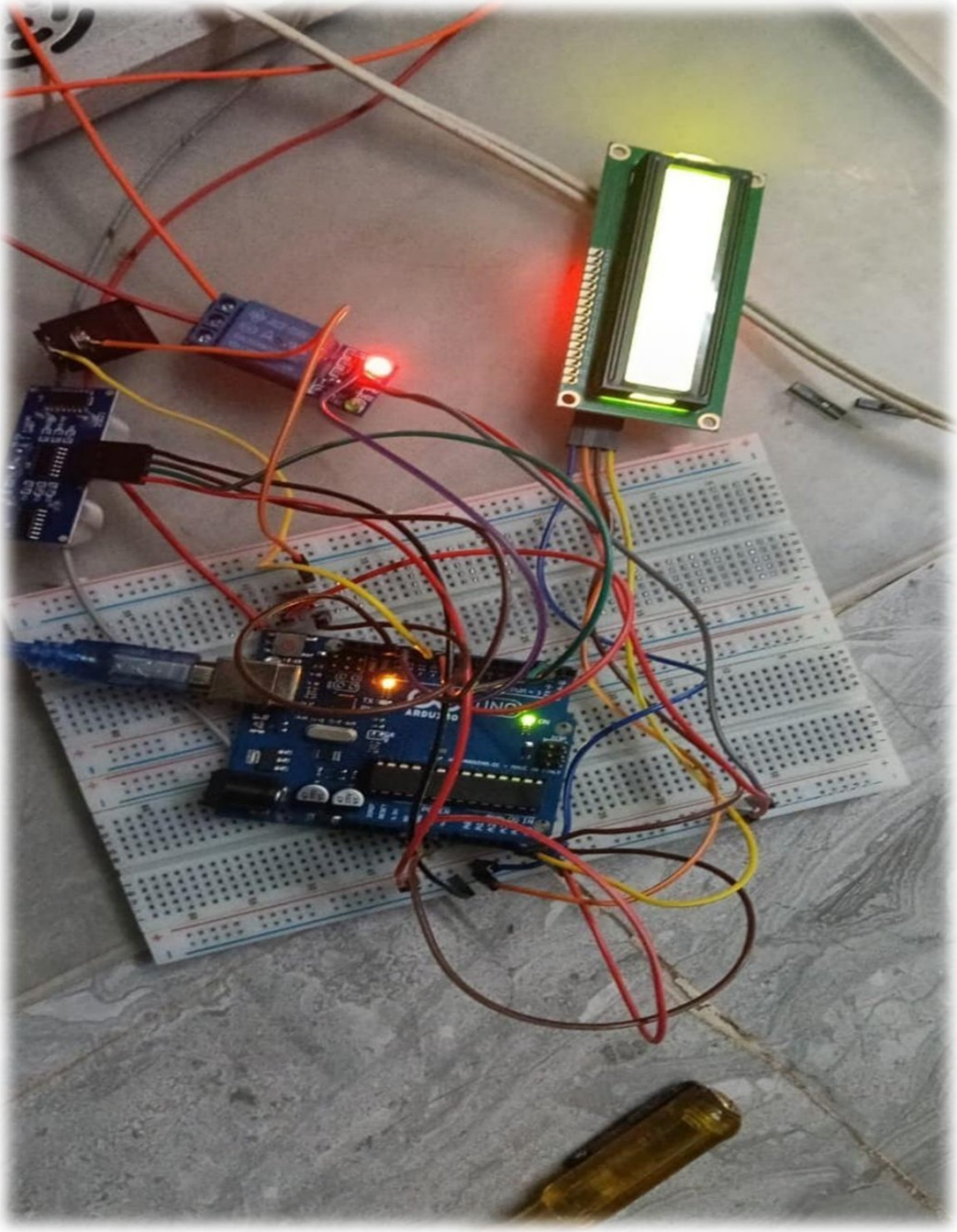


Figure :1 Working

2. Coding

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------- LCD ----------
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------- PIN CONNECTIONS ----------
const byte TRIG_PIN   = 2;
const byte ECHO_PIN   = 3;
const byte SET_PIN    = 10;
const byte MODE_PIN   = 11;
const byte BUZZER_PIN = 12;
const byte RELAY_PIN  = 13;

// ---------- TANK SETTINGS ----------
// Set this to the distance in cm from the
// sensor face to the tank bottom when EMPTY.
const float TANK_HEIGHT_CM = 20.0;

// Pump starts below this water percentage.
const int LOW_LEVEL = 30;

// Pump stops and buzzer alerts at this percentage.
```

```
const int HIGH_LEVEL = 85;


// Many relay modules are active LOW.
// Change to true if your relay turns ON with LOW.
const bool RELAY_ACTIVE_LOW = false;


// Set true for a passive buzzer.
// Set false for an active buzzer.
const bool BUZZER_PASSIVE = true;


// ---------- VARIABLES ----------
float distanceCm = 0;
int waterPercent = 0;


bool pumpOn = false;
bool manualMode = false;
bool sensorOK = false;


bool lastModeReading = HIGH;
bool lastSetReading = HIGH;
bool stableMode = HIGH;
bool stableSet = HIGH;


unsigned long modeChangedAt = 0;
unsigned long setChangedAt = 0;
```

```
unsigned long lastSensorRead = 0;
unsigned long lastDisplay = 0;
unsigned long lastReport = 0;

const unsigned long DEBOUNCE_MS = 40;

// ---------- PUMP CONTROL ----------
void setPump(bool on) {
  pumpOn = on;

  if (RELAY_ACTIVE_LOW) {
    digitalWrite(RELAY_PIN, on ? LOW : HIGH);
  } else {
    digitalWrite(RELAY_PIN, on ? HIGH : LOW);
  }
}

// ---------- BUZZER CONTROL ----------
void setAlarm(bool on) {
  if (on) {
    if (BUZZER_PASSIVE) {
      tone(BUZZER_PIN, 3000);
    } else {
      digitalWrite(BUZZER_PIN, HIGH);
    }
```

```
  } else {
    noTone(BUZZER_PIN);
    digitalWrite(BUZZER_PIN, LOW);
  }
}

// ---------- SENSOR READING ----------
bool readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(3);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration =
      pulseIn(ECHO_PIN, HIGH, 30000UL);

  if (duration == 0) {
    return false;
  }

  float reading = duration * 0.0343 / 2.0;

  // Reject readings outside the expected tank range.
```

```
  if (reading < 2.0 ||
      reading > TANK_HEIGHT_CM + 5.0) {
    return false;
  }

  distanceCm = reading;

  float level =
      (TANK_HEIGHT_CM - distanceCm) * 100.0
      / TANK_HEIGHT_CM;

  waterPercent = constrain((int)(level + 0.5), 0, 100);

  return true;
}

// ---------- LCD UPDATE ----------
void showLCD() {
  lcd.setCursor(0, 0);

  if (!sensorOK) {
    lcd.print("SENSOR ERROR    ");
    lcd.setCursor(0, 1);
    lcd.print("CHECK SENSOR    ");
    return;
```

```
  }

  lcd.print("WATER:");
  if (waterPercent < 100) lcd.print(" ");
  if (waterPercent < 10) lcd.print(" ");
  lcd.print(waterPercent);
  lcd.print("%     ");

  lcd.setCursor(0, 1);
  lcd.print("PUMP:");
  lcd.print(pumpOn ? "ON " : "OFF");

  lcd.print(" ");
  lcd.print(manualMode ? "MANUAL" : "AUTO  ");
}

// ---------- SERIAL MONITOR ----------
void showSerial() {
  Serial.print("Sensor: ");
  Serial.print(sensorOK ? "OK" : "ERROR");

  if (sensorOK) {
    Serial.print(" | Distance: ");
    Serial.print(distanceCm, 1);
    Serial.print(" cm | Water: ");
```

```
    Serial.print(waterPercent);
    Serial.print("%");
  }

  Serial.print(" | Pump: ");
  Serial.print(pumpOn ? "ON" : "OFF");

  Serial.print(" | Mode: ");
  Serial.print(manualMode ? "MANUAL" : "AUTO");

  Serial.print(" | Buzzer: ");
  Serial.println(sensorOK && waterPercent >= HIGH_LEVEL
                 ? "ON" : "OFF");
}

// ---------- SETUP ----------
void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(SET_PIN, INPUT_PULLUP);
  pinMode(MODE_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RELAY_PIN, OUTPUT);
```

```
  // Keep pump and buzzer OFF during startup.
  setPump(false);
  setAlarm(false);

  Wire.begin();

  lcd.init();
  lcd.backlight();
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("WATER TANK");
  lcd.setCursor(0, 1);
  lcd.print("SYSTEM STARTING");

  Serial.println("WATER TANK MONITOR");
  Serial.println("Serial speed: 9600 baud");
  Serial.println("D11: Auto/Manual");
  Serial.println("D10: Manual pump toggle");

  delay(1500);
  lcd.clear();
}
```

```
// ---------- MAIN LOOP ----------
void loop() {
  unsigned long now = millis();

  // Read sensor every 250 ms.
  if (now - lastSensorRead >= 250) {
    lastSensorRead = now;

    sensorOK = readDistance();

    if (!sensorOK) {
      // Fail-safe: stop pump if sensor data is invalid.
      setPump(false);
      setAlarm(false);
    }
  }

  // ---------- MODE BUTTON: D11 ----------
  bool modeReading = digitalRead(MODE_PIN);

  if (modeReading != lastModeReading) {
    modeChangedAt = now;
    lastModeReading = modeReading;
  }
```

```
if (now - modeChangedAt >= DEBOUNCE_MS &&
    modeReading != stableMode) {
  stableMode = modeReading;

  if (stableMode == LOW) {
    manualMode = !manualMode;

    // Prevent automatic state carrying into manual mode.
    setPump(false);

    Serial.print("Mode changed to: ");
    Serial.println(manualMode ? "MANUAL" : "AUTO");
  }
}

// ---------- SET BUTTON: D10 ----------
bool setReading = digitalRead(SET_PIN);

if (setReading != lastSetReading) {
  setChangedAt = now;
  lastSetReading = setReading;
}

if (now - setChangedAt >= DEBOUNCE_MS &&
    setReading != stableSet) {
```

```
  stableSet = setReading;

  if (stableSet == LOW && manualMode && sensorOK) {
    // In manual mode, press D10 to toggle pump.
    setPump(!pumpOn);

    Serial.print("Manual pump: ");
    Serial.println(pumpOn ? "ON" : "OFF");
  }
}

// ---------- AUTOMATIC PUMP CONTROL ----------
if (sensorOK && !manualMode) {
  if (waterPercent <= LOW_LEVEL) {
    setPump(true);
  }

  if (waterPercent >= HIGH_LEVEL) {
    setPump(false);
  }
}

// ---------- FULL TANK ALARM ----------
if (sensorOK && waterPercent >= HIGH_LEVEL) {
  setAlarm(true);
```

```
  } else {
    setAlarm(false);
  }

  // ---------- UPDATE DISPLAYS ----------
  if (now - lastDisplay >= 250) {
    lastDisplay = now;
    showLCD();
  }

  if (now - lastReport >= 1000) {
    lastReport = now;
    showSerial();
  }
}
```

REFERENCES

- Flutter Documentation – Flutter framework, widgets, UI development and application structure. Flutter Documentation
- Dart Documentation – Dart programming language, libraries and asynchronous programming concepts. Dart
- Flutter Material Library – Material Design widgets and UI components used in Flutter applications. Flutter API
- Firebase Documentation – Firebase services and their integration with Flutter applications. Flutter Docs
- Firebase Authentication Documentation – User authentication and account management. Firebase
- Firebase Authentication with Flutter – Implementation of authentication in Flutter using Firebase. Firebase
- Firebase Password Authentication for Flutter – Email and password based registration and login. Firebase
- Firebase Firestore Documentation – Cloud database concepts, collections, documents and data management. Cloud Firestore Documentation
- Firebase Security Rules Documentation – Security and access control for Firebase data. Firebase Security Rules
- Flutter Packages / pub.dev – Packages and plugins used to extend Flutter application functionality. pub.dev